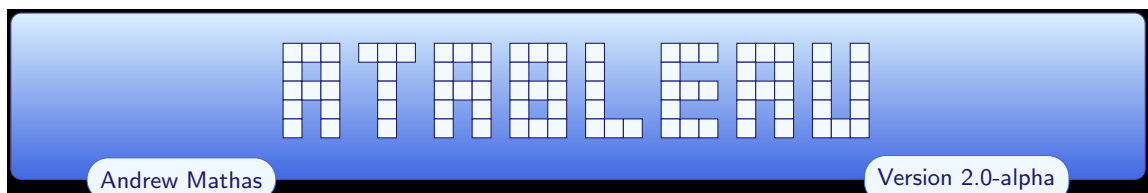




The package is already quite powerful, but it is still in the proof-of-concept phase. There are bugs and the documentation is incomplete. Several planned features have not been implemented and some things that have been implemented are still rough. Everything is subject to change.

The package will eventually be released on ctan. Comments and suggestions are welcome.



CONTENTS

1. INTRODUCTION

The commands are designed to be easy to use and easy to read. The literature contains many embellishments of the basic diagrams, so we provide mechanisms to customise and style these diagrams.

The package provides basic commands for drawing these objects, together with styling options and a key-value interface for adding extra features. Under the hood, everything is drawn using [TikZ](#). It is possible to use all of our commands either as standalone commands, or as components of a [tikzpicture](#) environment. This makes it possible to use these commands inside more complicated diagrams.

`\usepackage[options]{atableau}`

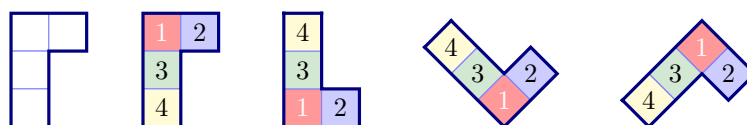
2. YOUNG DIAGRAMS AND TABLEAUX

Partitions, which are weakly decreasing sequences of non-negative integers, are fundamental objects in algebraic combinatorics and representation theory. Rather than working with sequences of integers, it is often useful to visualise partitions as Young *diagrams*, which are arrays of *boxes*, or *nodes*, in the plane. A *tableau* is a (Young) diagram where the boxes are labelled using some alphabet. Equivalently, a diagram is a tableau with empty labels.

For example, $\lambda = (4, 3, 3, 2, 0, 0, \dots)$ is a partition of 12, since the entries sum to 12. It is customary to omit the zeros when writing partitions and to use exponents for repeated parts, so we also write partitions in *exponential notation* as $\lambda = (4, 3^2, 2)$. Many authors identify a partition $\lambda = (\lambda_1, \lambda_2, \dots)$ with its *diagram*, which is the set

$$[\lambda] = \{ (r, c) \mid 1 \leq c \leq \lambda_r \text{ for } r \geq 1 \}.$$

The diagram of λ is visualised as a justified array of boxes in the plane, where the first row has 4 boxes, the next two rows have 3 boxes and the last row has 2 boxes. The *shape* of a diagram is the partition that gives the number of boxes in each row. Here are some examples of the diagrams and tableaux of shape $(2, 1^2)$ that can be drawn using [aTableau](#):



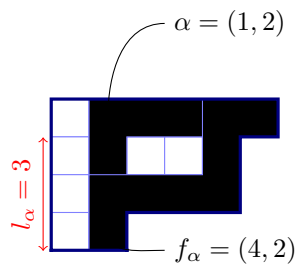
This choice of colouring is gratuitous, but it emphasises that [aTableau](#) allows you to add style (and, in particular, colour), to every box in these diagrams. This example also shows that [aTableau](#) supports the four different conventions used for drawing Young diagrams and tableaux. The following table shows how to draw the Young diagram of shape $\lambda = (4, 3^2, 2)$ using the four tableaux conventions supported by [aTableau](#):

Convention	aTableau command	Diagram
English	<code>\Tableau[english]{1234,567,89{10},{11}{12}}</code> (the default)	
French	<code>\Tableau[french]{1234,567,89{10},{11}{12}}</code>	
Ukrainian ¹	<code>\Tableau[ukrainian]{1234,567,89{10},{11}{12}}</code>	
Australian ²	<code>\Tableau[australian]{1234,567,89{10},{11}{12}}</code>	

The different conventions for diagrams, and tableaux, are listed in order of (my perception of) their popularity in the literature. By default, the *English* convention is used, so `\Tableau{1234,567,89{10},{11}{12}}` produces the first tableau above. These four conventions are used in the same way with the `\Diagram` command, which produces Young diagrams. You can change the default convention using the `\aTabset` command.

This table shows how the commands provided by aTableau work: there are basic commands for drawing diagrams, and these diagrams can be embellished using optional arguments and a key-value interface. In addition, as the following example shows, by supplying (optional) Cartesian coordinates, these diagrams can be used inside a `tikzpicture` environment to create more exotic pictures:

```
% \usetikzlibrary{patterns}
\begin{tikzpicture}[
  B/.style={pattern=north east lines, pattern color=blue,opacity=0.5},
  G/.style={pattern=north west lines, pattern color=ForestGreen,opacity=0.5},
]
\Diagram(0,0)[ribbons={ (G)16ccccrrr, (B)16crrcccr }]{6,5^2,2}
\draw[->](1.5, 1)node[right]{  $\alpha=(1,2)$  } to [out=180,in=90] (B-1-2.base);
\draw[->](1.5,-2)node[right]{  $f_\alpha=(4,2)$  } to [out=180,in=270] (B-4-2.base);
\draw[red,<->]([xshift=-1mm]B-1-1.south west)
  --node[rotate=90,above]{  $l_\alpha=3$  }([xshift=-1mm]B-4-1.south west);
\end{tikzpicture}
```



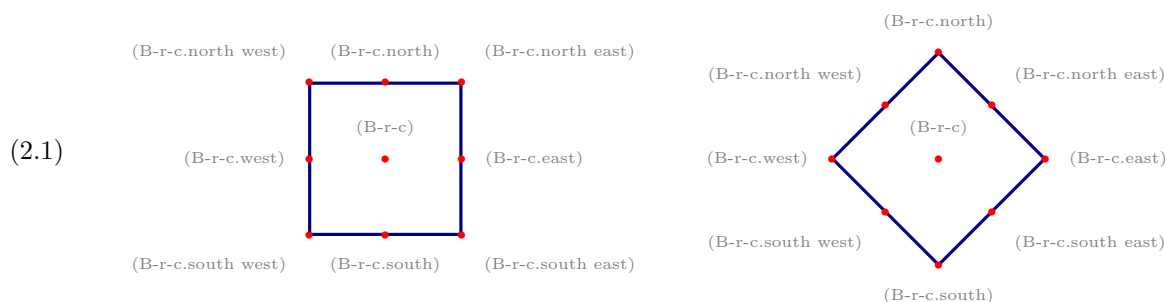
This example shows that partitions can either be typed out in full, or using exponential notation, which is usually shorter and easier to read.

¹Some authors call this the Russian convention

²I have not seen this convention in the literature, but this name for the convention seems apt.

Most of the commands in this example are [TikZ](#) commands, but most of the picture is drawn by the [\Diagram](#) command. The top of the example defines two [TikZ](#)-styles, B and G, which are used in the ribbons. The Young diagram is placed at position (0,0) by the [\Diagram](#) command. The three [\draw](#) commands in this example are used to add the three arrows to the picture. You should consult the very readable [TikZ](#) manual for an explanation of how these commands work.

From the viewpoint of [aTableau](#), the most important point of this example is that the [\Diagram](#) command can be used inside a [tikzpicture](#) environment because it is equipped with the Cartesian coordinate (0,0). The example also uses the four named coordinates, (B-1-1), (B-1-2), (B-4-1), and (B-4-2). When [aTableau](#) draws a tableau, or a diagram, it creates [TikZ](#) named coordinates for each of the boxes in the diagram, so that (B-r-c) refers to the box in row r and column c. Giving finer control, the following standard [TikZ](#) anchors are available:



In the name name (B-r-c), *r* and *c* should be replaced by the row and column index of a box in the tableau, because these anchors are *only* defined for boxes in the tableau. These *anchors* are a standard part of [TikZ](#). If you change the shape of the node then the available anchors may change; for more information see the [TikZ](#) manual. You can change the B in the name of these anchors to anything you like using the name option.

The previous example also shows that the [\Diagram](#) command accepts a *ribbons* option, which adds one or more *ribbons* to the diagram. Both the ribbon and the individual boxes in the ribbon can be given [TikZ](#) styling; see [??](#). All of the options to these commands are described in the following sections.

In the example above the Cartesian coordinate (0,0) is necessary because the [\Diagram](#) command is embedded inside a [tikzpicture](#) environment, but the coordinate itself is not important in the sense that changing the coordinate will not change the picture. Cartesian coordinates become more meaningful when there are more components inside the [tikzpicture](#) environment. Consider:



In particular, in the first diagram, notice the use of `name=A` to make the named coordinates take the form (A-r-c), instead of the default of (B-r-c).

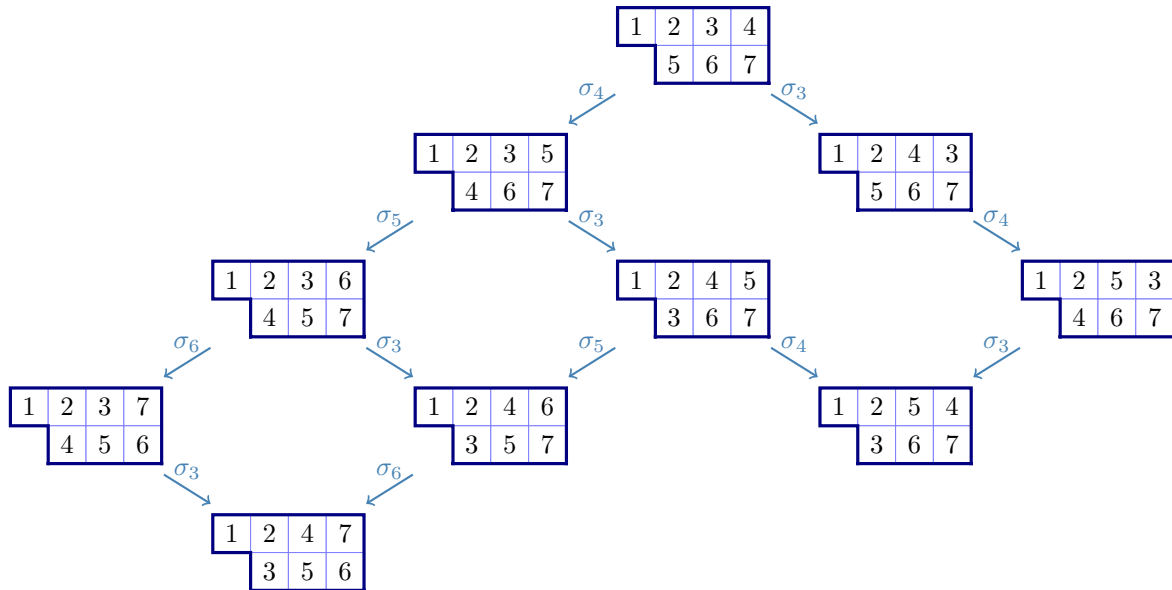
The following sections describe the commands defined by the package and how to use them. We start with the [\Tableau](#) command because all of the diagram and tableau commands, except for [\RibbonTableau](#), are derived from [\Tableau](#).

Finally, the condition that coordinates are required only when these diagrams are being used inside a [tikzpicture](#) environment needs to be interpreted appropriately. For example, coordinates are not required when the diagrams are inside nodes in these diagrams:

```

% \usetikzlibrary{matrix}
\begin{tikzpicture}[arr/.style={->,SteelBlue, thick}]
  \matrix (M)[matrix of nodes,row sep=4mm,column sep=4mm]{
    & & & \ShiftdTableau{1234,567} & \\
    & & \ShiftdTableau{1235,467} & & \\
    & & \ShiftdTableau{1243,567} & & \\
    & \ShiftdTableau{1236,457} & & & \\
    & & \ShiftdTableau{1245,367} & & \\
    & & \ShiftdTableau{1253,467} & & \\
    \ShiftdTableau{1237,456} & & & & \\
    & & \ShiftdTableau{1246,357} & & \\
    & & \ShiftdTableau{1254,367} & & \\
    & \ShiftdTableau{1247,356} & & & \\
  };
  \draw[arr] (M-1-4) --node[above]{$\sigma_4$} (M-2-3);
  \draw[arr] (M-1-4) --node[above]{$\sigma_3$} (M-2-5);
  \draw[arr] (M-2-3) --node[above]{$\sigma_5$} (M-3-2);
  \draw[arr] (M-2-3) --node[above]{$\sigma_3$} (M-3-4);
  \draw[arr] (M-2-5) --node[above]{$\sigma_4$} (M-3-6);
  \draw[arr] (M-3-2) --node[above]{$\sigma_6$} (M-4-1);
  \draw[arr] (M-3-2) --node[above]{$\sigma_3$} (M-4-3);
  \draw[arr] (M-3-4) --node[above]{$\sigma_5$} (M-4-3);
  \draw[arr] (M-3-4) --node[above]{$\sigma_4$} (M-4-5);
  \draw[arr] (M-3-6) --node[above]{$\sigma_3$} (M-4-5);
  \draw[arr] (M-4-1) --node[above]{$\sigma_3$} (M-5-2);
  \draw[arr] (M-4-3) --node[above]{$\sigma_6$} (M-5-2);
\end{tikzpicture}

```



2A. Tableaux. This section describes the `\Tableau` command, which draws tableaux or labelled diagrams. The syntax for this command is:

`\Tableau (x,y) [options] {tableau entries}`

(x,y) :
The Cartesian coordinates (x,y) are needed if and only if the tableau is inside a `tikzpicture` environment.

options :
Optional arguments, described below.

tableau entries :
The `tableau entries` are given as a comma separated list, with commas separating rows and each

token, or braced-group of tokens, being in a separate column. As we explain below, each tableau entry can be prefixed by (optional) **TikZ** styling commands.

We show by example how to use the `\Tableau` command, starting with basic usage and finishing with style. Inside `\Tableau`, the rows are separated by commas, with the column entries given by the “letters” in the entry for each row:

1	2	3	4
5	6		
7			
8			

α	γ	δ	q
x	y	η	
λ	μ		
π	Σ		

```
\Tableau{ 1234, 56, 7, 8 }
\Tableau{ \alpha\gamma\delta q,
          xy\eta,
          \lambda\mu,
          \pi\sum }
```

2A.1

You can put all of the tableau specifications on one line, without any spaces. Alternatively, as I do, you can add spaces and line breaks between the tableau entries to improve readability (although, you cannot have blank lines inside a `\Tableau` command). As this example suggests, by default, the entries of a tableau are typeset as mathematics, but this can be changed.

As the second example above shows, the tableau entries are *not* just individual characters: they also include $\text{\LaTeX}$ commands. Tableaux frequently contain entries that are not single characters, or given by commands. Such entries can be added to a tableau by enclosing them in braces:

1	3	10	11	2x
2	x^2			

```
\Tableau{ 13{10}{11}{2x}, 2{x^2} }
```

2A.2

There is one additional caveat for braced-entries: any *singleton* braced entry `{...}` in a row needs to be double-braced. Double-bracing is only necessary when you have a row with a single column and the entry in that column needs braces. The need for double-bracing of singleton entries is a consequence of how $\text{\LaTeX}$ 3 detects braced commas in comma separated lists.

The following example shows the difference between single and double-bracing for tableau entries.

1	3	4	5
8	10		
1	1		
1	2		

1	3	4	5
8	10		
11			
12			

```
\Tableau{ 1345, 8{10}, {11}, {12} }
\Tableau{ 1345, 8{10}, {{11}}, {{12}} }
```

2A.3

The 10's in these tableaux do not need to be doubled-braced because they are not the only entry in the second row. In contrast, the 11 and 12 in the first tableau are both treated as being two characters in different columns in the tableau because they are not doubled-braced. In the second tableau, the 11 and 12 occupy only one column because they are double-braced.

2A.1. *Adding style.* The **aTableau** package provides several different ways to add **TikZ** style specifications to the entries in tableaux, and to boxes in diagrams. This is made possible by adding style prefixes to the tableau entries. To take advantage of these styles you will need at least rudimentary knowledge of how **TikZ**-styles work. The examples in this manual give a good indication of what is possible. For anything more exotic, please consult the **TikZ** manual.

The simplest way to change the style of an entry in a tableau is to add a `*`-prefix to the entry in the tableau specifications.

1	2	3	4	5
6	7	8	9	
10	11	12		
13				

```
\Tableau{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

2A.4

By default, the `*`-entries are given the **TikZ** styling `fill=blue!20`. You can change the default `*`-style using the `star` style key:

1	2	3	4	5
6	7	8	9	
10	11	12		
13				

```
\Tableau[star style={fill=red!20,draw=red,thick}]
{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

2A.5

Notice that the `*`-styling takes precedence over the default styling of the surrounding boxes. This is because the boxes with the default styling are placed first, in the order that they are entered, after which the boxes with custom stylings are placed.

The `*`-syntax is a quick shorthand for adding emphasise some of the boxes in a tableau using a *single* style. It is possible to give every box in the tableau its own style by putting `TikZ`-style specifications inside square brackets, `[...]`, before the corresponding letter in the tableau specification. You can mix the `*`-syntax and the `[...]`-style syntax for different entries, although you can only apply one of these style settings to any given box.

13				
10	11	12		
6	7	8	9	
1		3		

```
\Tableau[french]{
  1 [blue!20]2 [circle]3 [red]4 [cyan]5,
  6 *7 8 9,
  {10} *{11} {12},
  {{13}} }
```

2A.6

It is not necessary to add spaces between the entries like this. This example puts spaces between each entry to make it clearer how the style specifications are applied to the boxes in the tableau.

Omitting some technicalities, each box in a tableau is constructed as a `TikZ` node, so any `TikZ`-style settings for a node can be used. This example shows that some care is needed when changing the style because simply giving a colour will change both the colour used to fill the box and the text colour, which is why some of tableau entries in the last example appear to be blank. If you are already familiar with `TikZ` then you probably already know what to do. If not, then here is a short list of useful style specifications for `TikZ` nodes:

Style	Meaning
<code>text=⟨colour⟩</code>	Sets the text colour in the box
<code>fill=⟨colour⟩</code>	Sets the fill, or background, colour of the box
<code>draw=⟨colour⟩</code>	Sets the boundary colour of the box

In the style settings, any valid `LATEX` colour name can be used; see, for example, the `xcolor` manual. In addition, you can change the border thickness, and its properties, with the style specifications `thin`, `thick`, `very thick`, `ultra thick`, `dashed`, `dotted`,

If you want to apply “complicated” style settings, such as `draw=cyan,ultra thick` which consists of a comma separated list of `TikZ`-style settings, then the entire style must be put inside matching square and curly braces `[{...}]`. This is necessary to ensure that `LATEX` does not get confused by the commas in the style setting because commas are used to separate the rows of the tableau.

1	2	3	4	5
6	7	8	9	
10	11	12		
13				

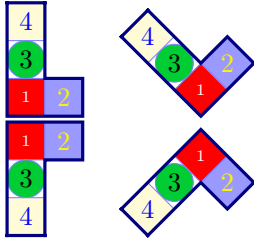
```
\Tableau[australian]{
  1 [fill=blue!20]2 3 [text=red]4 5,
  [{circle,fill=orange}]6 *7 8 9,
  [{draw=cyan,ultra thick}]{10} *{11} {12},
  [{dashed,draw=red,ultra thick}]{13} }
```

2A.7

The last example shows that it is not necessary to double-brace singleton entries when they have a style specification.

This example shows that modifying the borders of a box does not add much emphasis, so use this sparingly, especially because any boxes placed later are likely to redraw the boundary (see [Subsection 2A.5](#)). In addition, the orange circle in this example looks too big, but it is what we asked for because its diameter matches the horizontal width of the box. The next example shows that we get a better result for *Australian* and *Ukrainian* tableaux if we add a *minimum size* specification.

For complicated styles, or styles that are used often, we recommend defining the style outside of the `\Tableau` command by using the `\tikzset` command:



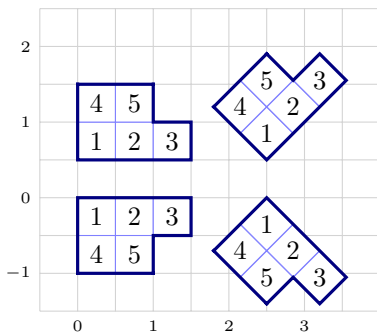
```

\tikzset{
  B/.style={fill=blue!40, text=yellow},
  G/.style={fill=green!80!blue, circle, minimum size=5mm},
  R/.style={fill=red, text=white, font=\tiny},
  Y/.style={fill=yellow!20, text=blue}
}
\Tableau[french]{[R]1[B]2,[G]3,[Y]4}
\Tableau[ukrainian]{[R]1[B]2,[G]3,[Y]4}
\Tableau[english]{[R]1[B]2,[G]3,[Y]4}
\Tableau[australian]{[R]1[B]2,[G]3,[Y]4}

```

In this example, the `TikZ`-styles `B`, `G`, `R`, and `Y` are applied to the tableaux entries labelled 1, 2, 3, and 4, respectively. The colours used here, and throughout this manual, are for demonstration purposes only: I recommend against such garish choices in any self-respecting document!

2A.2. Tableau coordinates. The (x, y) -coordinates are needed if, and only if, `\Tableau` command is used inside a `tikzpicture` environment, in which case (x, y) gives the coordinates of the “outside corner” of the box in row 1 and column 1 of the tableau. The following example shows how tableaux are placed using (x, y) -coordinates inside a `tikzpicture` environment. The example also shows that, by default, the tableau boxes are half a unit wide.



```

\begin{tikzpicture}[addgrid]
\Tableau(0,0) [english] {123,45}
\Tableau(0,0.5) [french] {123,45}
\Tableau(2.5,0.5) [ukrainian] {123,45}
\Tableau(2.5,0) [australian] {123,45}
\end{tikzpicture}

```

2A.3. Tableau options. The optional `*-styles` and `[...]-styles` are the main mechanism for styling individual nodes in a tableau. Using a key-value syntax, you can change other aspects of the tableaux produced by the `\Tableau` command. The rest of this section describes these keys, their default values, and gives examples of how to use them.

The options described below can be used with the `\Tableau` command by placing them as a comma separated list inside square brackets:

`\Tableau[options] {tableau entries}` or `\Tableau(x,y) [options] {tableau entries}`.

The options can appear in any order, with later options having precedence over earlier ones.

When used inside a `\Tableau` command, the options only affect that particular tableau. Most of these options, or settings, can also be used in the `\aTabset` command, or as optional arguments in `\usepackage[...]{atableau}`, to change the default settings of all tableaux in the same $\text{\LaTeX}$ group. For example, if you put the command `\aTabset[ukrainian]` in the preamble of your document then, by default, latter tableaux and Young diagrams will be drawn using the Ukrainian convention.

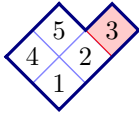
english (default: **english**)

french

ukrainian

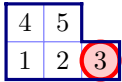
australian

These four options change the convention, or orientation that is used to draw the tableaux. Rather than define this precisely, we use the tired and true method in combinatorics of defining by example.



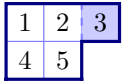
```
\tikzset{R/.style={fill=red!20,draw=red}}
\Tableau[ukrainian]{12[R]3,45}
```

2A.10



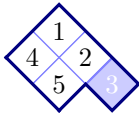
```
\tikzset{R/.style={fill=red!20, circle, draw=red, ultra
thick}}
\Tableau[french]{12[R]3,45}
```

2A.11



```
\tikzset{B/.style={fill=blue!20, dashed, thick}}
\Tableau[english]{12[{B, thick}]3,45}
```

2A.12



```
\tikzset{B/.style={fill=blue!20, thick, text=white}}
\Tableau[australian]{12[{B, thick}]3,45}
```

2A.13

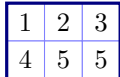
By default, the english convention is used to draw tableaux and Young diagrams. This can be changed using `\aTabset`. The capitalised names, `Australian`, `English`, `French` and `Ukrainian`, are also recognised. If, for example, your document is only going to french tableaux and diagrams, then you can specify this when you load the package using `\usepackage[french]{atableau}`.

The `\Tableau` settings used as options to the `\usepackage` statement change the default settings throughout the document.

align (default: `bottom`)

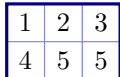
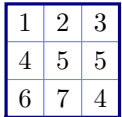
[accepts: `centre`/`bottom`/`top`]

Use the `align` key to change the vertical centering of tableaux. By default, tableaux are aligned on their baseline.



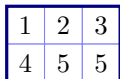
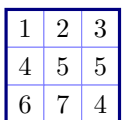
```
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.14



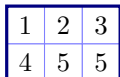
```
\Tableau[align=top]{123,455,674}
\Tableau[align=top]{123,455}
```

2A.15



```
\aTabset{align=bottom}
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.16



```
\aTabset{align=centre}
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.17

It is not necessary to have the same `align` keys on each tableau. Here, and elsewhere, American spelling variations are tolerated.

box height (default: `0.5`)

[accepts: A decimal giving the height in cm]

box width (default: `0.5`)

[accepts: A decimal giving the width in cm]

Sets the width and height of the boxes in the tableau. The default height and width of the boxes is 0.5 cm when using the english and french conventions and 0.7012 cm, which is approximately $1\sqrt{2}$ cm, when using the ukrainian and australian conventions. (In all cases, the side length of the squares is 0.5 cm.)

conjugate (default: `false`)

[accepts: `false`/`true`]

Draws the *conjugate* tableau, where the rows and columns are swapped.

aTableau

1	2	3
4	5	
6	7	

1	4	6
2	5	7
3		

```
\Tableau{123,45,67}
\Tableau[conjugate]{123,45,67}
```

2A.18

name (default: B)

[accepts: string/character]

By default, the boxes can be referenced using the node names (B-1-1), (B-1-2), (B-1-3), ..., with (B- r - c) referring to the node in row r and column c . This feature is most useful when you have several tableaux inside a `tikzpicture` environment because if you only have one tableau, then it is unlikely that you need to change the default prefix for the node names. Examples are given in ?? and ??.

See the introduction to this section for examples and for more information about using node names and anchors.

tikz before
tikz after

[accepts: **TikZ** commands]

[accepts: **TikZ** commands]

These commands inject **TikZ** code into the underlying `tikzpicture` environment, where the tableau is constructed, before and after the `\Tableau` command, respectively.

	1	3	4	
	5	6	7	
	8	9		

```
\Tableau[
  tikz before={\draw[help lines,step=0.5](-0.5,-2)grid(2,0.5);},
  tikz after={\draw[ultra thick,red]
    (B-1-3.north east)--(B-1-3.south east)
    --(B-2-1.north east)--(B-3-1.south east)
    --(B-3-1.south west);
  }]{134,567,89}
```

2A.19

Both before and after hooks are provided because the order in which you place **TikZ** commands can affect the drawing, primarily because later commands are drawn over the top of earlier ones. Note that the *before* code cannot use the named nodes B- r - c because this code is executed before the tableau is drawn. For examples, see Subsection 2A.5.

tikzpicture

[accepts: **TikZ**-style commands]

Use this command to add an optional argument to the underlying `tikzpicture` environment that contains the tableau. The `tikzpicture` option is ignored when the `\Tableau` command is equipped with (x,y) -coordinates.

13	11	12	9	5
10	7	8	4	
6	1	2	3	

```
\Tableau[ukrainian, name=A,
  tikzpicture={remember picture}]
{ 1[fill=blue!20]23[text=red]45,
  6*78[{\draw=cyan,ultra thick}]9,
  [{circle,fill=orange}]{10}*{11}{12},
  [{dashed,draw=red,ultra thick}]{13}
}
```

2A.20

Since `name=A`, the nodes in this tableau are referenced as (A-1-1), (A-1-2), (A-1-3), As the example also uses `tikzpicture=remember picture`, we can reference the node names in last tableau from other `tikzpicture` environments. For example, we can do things like this:

This circle is too big!
Use minimum size=5mm

```
\tikz[remember picture, overlay]
\draw[very thick, ->,red]
(0,0) node[right,align=left]
{This circle is too big!\Use minimum size=5mm}
to [out=135, in=225] (A-3-1);
```

2A.21

As a second example, we can use `tikzpicture=rotate=30` to try to rotate a tableau by 90 degrees with:

aTableau

3	
2	5
1	4

3	
2	5
1	4

```
\Tableau[tikzpicture={rotate=90}]{123,45}
\begin{tikzpicture}[rotate=90]
  \Tableau(0,0){123,45}
\end{tikzpicture}
```

2A.22

The two tableaux constructions in this example are equivalent. An oddity with both of these examples is that the numbers have not been rotated, which is because *rotate* does not affect component objects, such as nodes, inside a **TikZ** picture. You can rotate everything in the picture by using *transform canvas*:

3	
2	5
1	4

```
\Tableau[tikzpicture={transform canvas={rotate=90}}]{123,45}
```

2A.23

inner wall

outer wall

inner style

outer style

fill

text

font

shape

math nodes (default: **true**) [accepts: true/false] **text nodes** (default: **false**) [accepts: true/false]

ribbons [accepts: a comma separated list of ribbon specifiers] **snobs** [accepts: a comma separated list of ribbon specifiers] **ribbon style** (default: **fill=blue!20**) [accepts: **TikZ**-style commands]

Use the **ribbons** key to add a comma separated list of ribbons to your tableau. The syntax for adding ribbons is described in [Section 2F](#), which describes the **\RibbonTableau** command. The basic idea is that you specify the location of the head of the ribbon, which has minimum row index and maximum column index, and then gives a sequence of *r*'s and *c*'s depending on whether the ribbon moves along a row or column – and, of course, you can add style. Use the **ribbon style** option to change the default style of the ribbons. For example, noting that a box is a ribbon of length 1, the following code uses ribbons of length one (that is, nodes), to highlight the addable nodes of this tableau.

1	2	3	4	
5	6	7		
8	9	10		
11				

```
\Tableau[star style={fill=green!80!blue,text=white},
  ribbon style={fill=yellow},
  ribbons={15,24,42,51},
]{123*4,567,89*{10},*{11}}
```

2A.24

The ribbons are always assumed to be contained inside the diagram of the tableau, so they will not change the boundary of the tableau. Use **snobs** to add ribbons to the tableau that are placed *after* the border of the diagram has been drawn. (Writing *ribbons* backwards gives *snobbir*.) The default style for a **snobs** ribbon is given by **ribbon style**.

1	2	3	4	
5	6	7		
8	9	10		
11				

```
\Tableau[star style={fill=green!80!blue,text=white},
  ribbon style={fill=yellow},
  snobs={15,24,42,51},
]{123*4,567,89*{10},*{11}}
```

2A.25

The difference between these two pictures is that in the second picture the nodes in the **snobs** are drawn over the tableau boundary, causing the boundary to disappear for these nodes. For this reason, in most

cases you should use ribbons, and avoid snobs. I do not know of a real example where I would use snobs, so I would be pleased to hear of any example where it is necessary.

star style

box height (default: 0.5) a decimal number (in centimetres) **box width** (default: 0.5) a decimal number (in centimetres) **scale** (default: 1) [accepts: a decimal number] **xscale** (default: 1) [accepts: a decimal number] **yscale** (default: 1) [accepts: a decimal number]

By design, boxes in tableaux do not automatically resize to fit their contents. For example, consider:

1	3	10	11	$1+x$
2	$1+x$			

```
\Tableau{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.26

This is unlikely to be the desired output! The options `scale`, `xscale` and `yscale` can be used to rescale the boxes in tableaux and diagrams. The names are slightly misleading because `xscale` rescales in the direction of increasing column index and `yscale` rescales in the direction of increasing row index.

1	3	10	11	$1+x$
2	$1+x$			

```
\Tableau[scale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.27

1	3	10	11	$1+x$
2	$1+x$			

```
\Tableau[xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.28

				$1+x$
	$1+x$		10	11
2		3		
	1			

```
\Tableau[ukrainian, xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.29



The options, `scale`, `xscale`, and `yscale`, affect all `aTableau` diagrams, including tableaux, Young diagrams and abacuses. If you want to use `\aTabset` to change the sizes of all diagrams and tableaux in your document it may be better to use the `box height` and `box width` options.

.	.	
.		
.		

```
\Tableau[star style={blue}]{...*, *.*, .*}
```

2A.30

1	2	3	1	2	3
4	5		4	5	

```
\tikzset{R/.style={fill=blue!20,draw=red,thick}}
\Tableau{12[R]3,4[R]5}
\Tableau[star style={fill=blue!50,text=white}]{1*23,4*5}
```

2A.31

2A.4. *Compositions.* As we have defined them, tableaux are always of *partition* shape, in that the lengths of the rows form a weakly decreasing sequence. In fact, the `\Tableau` command also accepts tableaux of *composition* shape, where the lengths of the rows are not necessarily in decreasing order

2	4	5				2	4	5			
8	2	7	5	1			8	2	7		
3	1					2	3	1			

```
\Tableau{ 245,82751,31 }
\SkewTableau{1,2}{ 245,827,231 }
```

2A.32

Empty rows in tableaux and Young diagrams are not supported!

2A.5. *Drawing order.* When `TikZ` constructs a picture, later objects are drawn over the top of earlier ones, which means that earlier features can be obscured or lost. For this reason it is useful to know the drawing order used by :

- First, the tableau entries that use the default styling are placed, in the order that appear in the *tableau specifications*.
- Secondly, the styled tableau entries are placed, in the order that they appear in the *tableau specifications*.
- Next, the nodes in any ribbons are placed, again in the order that the ribbons are listed. For each ribbon, first the border of the ribbon is drawn, together with any styling for the ribbon, and then the boxes in the ribbon, together with any styling, are added.
- The boundary of the tableau is drawn next.
- Finally, the snobs are drawn, in the order that they are specified.

2A.6. *Special characters.* The characters `[`, `,`, and `*` have special meanings in the *tableau specifications*. If you want these characters to be entries in a tableau, then you need to enclose them in braces:

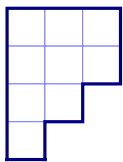
1	3	,	5	]
8	[			
*				

```
\Tableau{ 13{,}5], 8{[}, {{*}} }
```

2A.33

There is no need to enclose `]` in braces because it only becomes special when it is preceded by a `[` character.

2B. **Young diagrams.** A *Young diagram*, or simply a *diagram*, is an unlabelled tableau. Diagrams can be drawn using the `\Tableau` command:



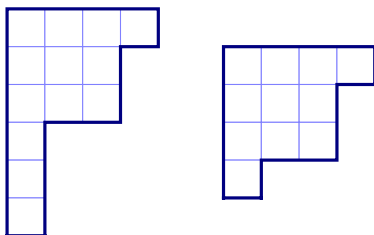
```
\Tableau{~~~,~~~,~~,~}
```

2B.1

This approach works but it is slightly cumbersome and hard to proofread, so `aTableau` provides the `\Diagram` command. The `\Diagram` command uses almost the same syntax as the `\Tableau` command:

```
\Diagram (x,y) [options] {partition}
```

As with the `\Tableau` command, the (x,y) -coordinates are needed if and only if the diagram is inside a `tikzpicture` environment. In the `\Diagram` command, partitions can be specified either as a comma separated list of weakly decreasing nonnegative integers, or using *exponential notation* for partitions:



```
\aTabset{align=centre}
\Diagram{4,3^2,1^3}
\Diagram{4,3,3,1}
```

2B.2

Internally, `\Diagram` actually does something like the first example in this section, so all of the options for the `\Tableau` command can be used with `\Diagram`. We do not repeat all of these options in this section and, instead, highlight some of the more interesting ones together with new options that apply specifically to diagrams.

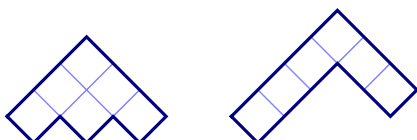
We start by advertising that `\Diagram` supports the four different conventions for diagrams, with `english` being the default:

`english` (default: `english`)

`french`

`ukrainian`

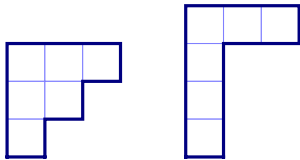
`australian`



```
\Diagram[australian]{3,2,1}
\Diagram[australian]{3,1^3}
```

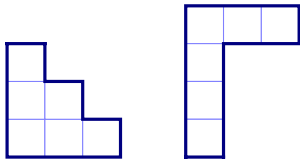
2B.3

aTableau



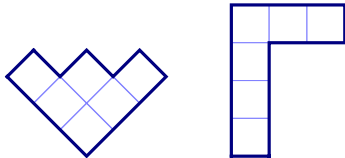
% English is the default convention
`\Diagram[english]{3,2,1}`
`\Diagram{3,1^3}`

2B.4



`\Diagram[french]{3,2,1}`
`\Diagram{3,1^3}`

2B.5



`\Diagram[ukrainian]{3,2,1}`
`\Diagram{3,1^3}`

2B.6

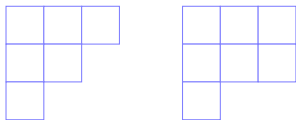
border (default: **true**)

[accepts: true/false]

no border (default: **false**)

[accepts: true/false]

If **border** is true, then the border of the tableau is drawn. If **border** is false, then the border of the tableau is not drawn. By default, the border is drawn. The **no border** option, which is provided only for completeness, is the inverse of **border**. That is, **no border** is equivalent to **border=false**, and **no border=false** is equivalent to **border=true**.



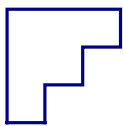
`\Diagram[border=false]{3,2,1}`
`\Diagram[no border]{3^2,1}`

2B.7

border only (default: **false**)

[accepts: false/true]

If **border only** is true, then only the border of the tableau is drawn, and not the internal walls around the boxes.



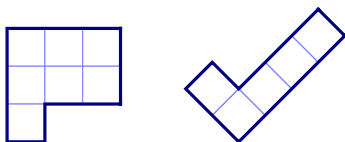
`\Diagram[border only]{3,2,1}`

2B.8

conjugate (default: **false**)

[accepts: false/true]

Draws the conjugate diagram, which has the rows and columns interchanged.



`\Diagram[conjugate]{3,2,2}`
`\Diagram[ukrainian, conjugate]{2,1^3}`

2B.9

ribbons

[accepts: ribbon specification]

snobs

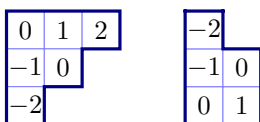
[accepts: ribbon specification]

To do: Add a example...

show [accepts: contents,final,hooks,initial,residue]

Use **show** option to draw particular types of tableau of the specified shape. The possible choices are:

- **show=contents** print the diagram where each box is labelled by its *content*, which is the row index minus the column index.

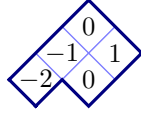


`\Diagram[show=contents]{3,2,1}`
`\Diagram[french,show=contents]{2^2,1}`

2B.10

- **show=final** print the *final tableau* of this shape, which has the numbers $1, 2, \dots, n$ entered in order from top to bottom down successive columns (with appropriate modifications for the non-English conventions).

1	4	7
2	5	
3	6	

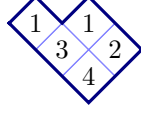


```
\Diagram[show=final]{3,2,2}
\Diagram[australian,show=contents]{2^2,1}
```

2B.11

- `show=hooks` print the diagram where each box is labelled by the corresponding *hook length*. If λ is a partition and λ' is its conjugate partition then the hook length of the box in row i and column j is $\lambda_i - i + \lambda'_j - j + 1$.

5	4	1
3	2	
2	1	

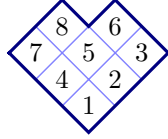


```
\Diagram[show=hooks]{3,2,2}
\Diagram[ukrainian,show=hooks]{2^2,1}
```

2B.12

- `show=initial` print the *initial tableau* of this shape, which has the numbers $1, 2, \dots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).

1	2	3
4	5	
6	7	

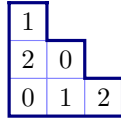


```
\Diagram[show=initial]{3,2,2}
\Diagram[ukrainian,show=initial]{3^2,2}
```

2B.13

- `show=residues` print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo some integer e , which must also be supplied.

0	1	0
1	0	
0		



```
\Diagram[show=residues, e=2]{3,2,1}
\Diagram[french,show=residues, e=3]{3,2,1}
```

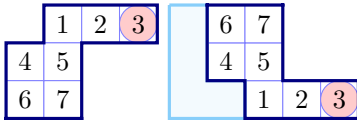
2B.14

2C. Skew tableau and skew diagrams. A partition μ is *contained* in a partition λ , which is written as $\mu \subseteq \lambda$, if $\mu_k \leq \lambda_k$, for $k \geq 0$. If $\mu \subseteq \lambda$ then the *skew partition* $\lambda/\mu = \{(r, c) \in \lambda \mid (r, c) \notin \mu\}$ is the set of nodes that are in λ and not in μ . In this manual, we call μ the *inner shape* and λ/μ the *outer shape*. A *skew tableau* is a labelling of the nodes in the diagram of a skew partition. Skew partitions and skew tableau can be drawn using the commands:

```
\SkewDiagram (x,y) [options] {inner shape} {outer shape}
\SkewTableau (x,y) [options] {inner shape} {skew tableau}
```

As with the previous commands, the (x, y) -coordinates should be given if and only if the picture is inside a `tikzpicture` environment. The inner and outer shapes can either be given as a comma separated list of integers or in exponential notation. In a `\SkewTableau`, the tableau entries are specified exactly as in the `\Tableau` command. In particular, they can be given optional style commands.

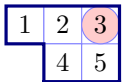
All of the options for the `\Diagram` command can be used with the `\SkewDiagram` command and, similarly, all of the options for the `\Tableau` command can be used with the `\SkewTableau` command.



```
\tikzset{R/.style={fill=red!20,circle}}
\SkewTableau{2,1^2}{12[R]3,45,67}
\SkewTableau[french,skewwall]{2,1^2}{12[R]3,45,67}
```

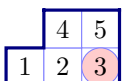
2C.1

2D. Shifted tableau and shifted diagrams. `\ShiftedDiagram (x,y) [options] {partition}`
`\ShiftedTableau (x,y) [options] {partition}`



```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau{12[R]3,45}
```

2D.1



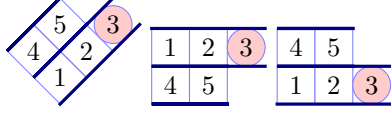
```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau[french]{12[R]3,45}
```

2D.2

2E. **Tabloids.** A *tabloid* is an equivalence class of tableau where two tableaux are equivalent if they have the same entries in each row, up to a permutation. In the literature, tabloids are usually drawn with lines above and below each row, and no side boundary walls. Tabloids can be drawn with the `\Tabloid` command.

`\Tabloid (x,y) [options] {partition }`

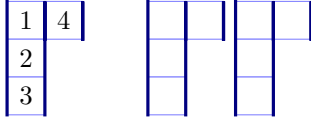
The `\Tabloid` command functions in the same way as the `\Tableau` command, except that it draws tabloids.



2E.1

```
\tikzset{R/.style={fill=red!20,circle,minimum size=5mm}}
\Tabloid[ukrainian]{12[R]3,45}
\Tabloid[english]{12[R]3,45}
\Tabloid[french]{12[R]3,45}
```

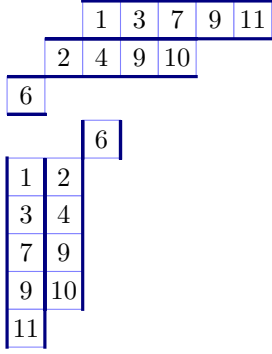
Some authors work with *column tabloids*. Such tableaux do not have a dedicated `aTableau` command, however, they can be drawn using the `conjugate` option.



2E.2

```
\Tabloid[conjugate]{123,4}
\Tabloid[conjugate]{~~~,~}
\Diagram[conjugate,tabloid]{3,1}
```

Similarly, skew tabloids can be drawn using the `skew` option.



2E.3

```
\aTabset{align=center}
\Tabloid[skew={2,1}]{1379{11},249{10},6,}
\Tabloid[conjugate,skew={1^2}]{1379{11},249{10},6,}
```

2F. **Ribbon tableaux.** A *ribbon* in a tableau, or in a diagram, is a connected strip of boxes R that are totally ordered by their contents. Recall from Section 2B that the content of (a, b) is $b - a$. Therefore, a ribbon does not contain a $\square$ -square, and if $(a, b) \in R$ then at most one of $(a + 1, b)$ and $(a, b - 1)$ is in R . The *head* of a ribbon R is the unique node $(a, b) \in R$ such that $a' > a$ and $b' < b$ whenever $(a', b') \in R$ and $(a', b') \neq (a, b)$.

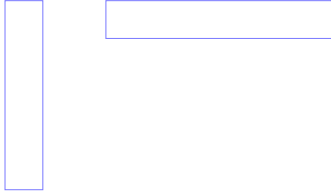
`\RibbonTableau (x,y) [options] {ribbon specifications}`

Like the other commands, the (x, y) -coordinates are optional and are required if and only if the diagram is being used as part of a `tikzpicture` environment. All of the options for the `\Tableau` command can be used for the ribbon command, so we refer to Section 2A for a description of the available options.

2F.1. *Ribbon specifications.* Unlike the `\Tableau` command, the entries of a ribbon tableau are given by ribbon specifications (rather than tableau) specifications. Ribbon specifications are also used in the `ribbons` and `snoobs` options discussed in Subsection 2A.3.

To understand the ribbon specifications, observe that if (a, b) is the head of a ribbon, then we can walk along the ribbon by specifying whether the row index increases or the column index decreases. That is, the ribbon is uniquely determined by specifying its head (a, b) together with a sequence of r 's and c 's to say whether the row index increases or the column index decreases. For example, vertical and horizontal ribbons are given by a sequences of r 's and c 's, respectively:

aTableau

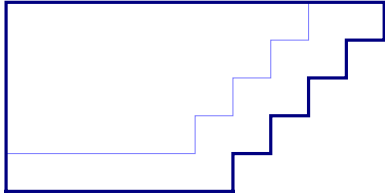


```
\aTabset{align=top, no border}
\RibbonTableau{11rrrr}
\RibbonTableau{16ccccc}
```

2F.1

The boundary of a ribbon tableau is the smallest (skew) partition that contains all of the ribbons. As the last example shows, the `no border` disables the tableau borders because, otherwise, we would not be able to see the ribbon boundary in this example because the ribbon is the full diagram.

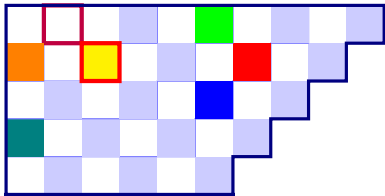
The following diagram shows a see-sawing ribbon with head in row 1 and column 10, and tail in row 5 and column 1.



```
\RibbonTableau{1{10}crcrcrcrccccc}
```

2F.2

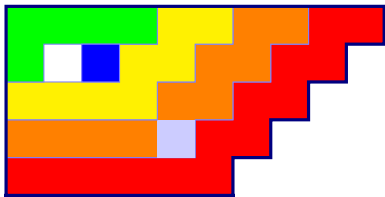
As with tableau entries, each box in a ribbon can be preceded with a style specification, either using `*` for a star style or using `[...]` for more customised style specifications.



```
\RibbonTableau{
  *1{10}c*rc*rc*rc*rc*cc*cc,
  *18c[fill=red]rc[fill=blue]rc*rc*cc[fill=teal]c,
  [fill=green]16c*rc*rc*cc,
  *14c[{draw=purple,ultra thick}]cc[fill=orange]r,
  [{fill=yellow,draw=red,ultra thick}]23
}
```

2F.3

In particular, the yellow box in this example highlights that boxes are ribbons of length 1. Most of the time, you want to style the entire ribbon rather than styling the individual boxes in the ribbon. Optionally, style for the entire ribbon can be given inside parentheses (...) at the *start* of the ribbon. Unlike the other style specifications, the `*`-shorthand for the star style cannot be applied to an entire ribbon.

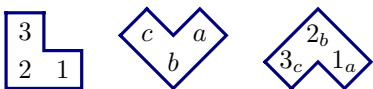


```
\RibbonTableau{
  (fill=red)1{10}crcrcrcrccccc,
  (fill=orange)18crcrc*rccccc,
  (fill=yellow)16crrc[cyan]ccc,
  (fill=green)14cccr,
  (fill=blue)23
}
```

2F.4

Any styles for a ribbon are applied to the region bounded by the closed path given by the boundary of the ribbon. The ribbon style is applied first, after which any styled boxes in ribbon are drawn. So, as this example shows, the optional style of any box in the ribbon takes precedence over the style of the full ribbon.

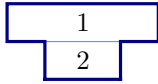
Finally, it is often necessary to label some of the boxes in the ribbon. Optionally, text can be added to the boxes in a ribbon by supplying it as a subscript to the corresponding entry in the ribbon specification.



```
\RibbonTableau[french] { 12_1 c_2 r_3 }
\RibbonTableau[ukrainian] { 12_a c_b r_c }
\RibbonTableau[australian]{ 12_{1_a} c_{2_b} r_{3_c} }
```

2F.5

This example adds spaces between the entries for the different boxes in the ribbon for clarity. There are situations where you might to put a label of a box on the boundary between two boxes in the diagram. There is no dedicated syntax for doing this, however, it is not difficult to do this using the named anchors of (2.1).



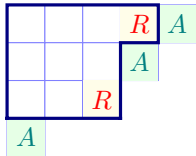
```
\RibbonTableau[shifted, tikz after={
  \node at (B-1-2.east){$1$};
  \node at (B-2-2.east){$2$};
}]{ 14ccc, 23c }
```

2F.6

We have now seen the full ribbon specifications. To summarise:

- The ribbon specification starts with optional **TikZ** styling for the entire ribbon, which is enclosed in (...).
- The first entry in the ribbon is given as **ab**, if the head of the ribbon is in row **a** and column **b**. The remaining entries in the ribbon are specified by a sequence of **r**'s and **c**'s, depending on whether the next box in the ribbon is given by increasing the row index, or decreasing the column index, respectively.
- Optionally, each box in the ribbon, including the head, can be given **TikZ**-styling by prefixing it with a *****, which gives the ribbon **star** style, or by giving **TikZ**-style commands inside square brackets [...].

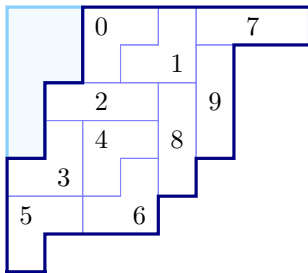
Finally, when drawing the border, ribbons in a `\RibbonTableau` are assumed to be contained in the smallest Young diagram that contains all of the ribbons. This said, it is up to you to ensure that the boxes in the ribbon not have negative row or column indices. Inside a `\Diagram` or `\Tableau` command, any ribbons added using `ribbons`, or `snoobs`, do not affect the border of the diagram or tableau, and it is up to you to ensure that the boxes in the ribbon are inside the diagram or tableau. This said, it is a useful feature to be able to place boxes outside of the diagrams using `ribbons` and `snoobs`. For example, the following picture uses `ribbons` to highlight the *addable* and *removable* nodes of the partition $(4, 3^2)$.



```
\tikzset{A/.style={fill=green!10, text=teal},
  R/.style={fill=yellow!10, text=red}}
\Diagram[ribbons={ [A] 15_A, [A] 24_A, [A] 41_A,
  [R] 14_R, [R] 33_R }]{4, 3^2}
```

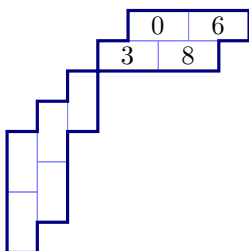
2F.7

We close this section showing how to draw some tableaux that appear in the literature.



```
\RibbonTableau[skew={2^2, 1^2}, skewwall]{
  14c_0r, 15r_1c, 18c_7c, 26r_9r,
  34c_2c, 35r_8r, 42r_3c, 44c_4r,
  54r_6c, 62c_5r
}
```

2F.8



```
\RibbonTableau[skew={4, 3, 2, 1}, scale=0.8,
  tikz after = {
    \foreach \r/\c/\l in {1/6/0, 1/8/6, 2/5/3, 2/7/8}
    { \node at (B-\r-\c.west){\l};}
  }
]{
  16c, 18c, 25c, 27c, 33r, 42r, 51r, 62r, 71r
}
```

2F.9